Mechanisms of Action (MoA) Prediction

Multi-label Drug Classification with Deep Learning

Kasra Korminejad

Stats718 Final Project

April 2026

Outline

1. Background
2. Dataset
3. Data Processing
4. Evaluation Metric
5. Models
6. Results
7. Takeaways & Future Work
8. Conclusion

The Competition

LISH-MoA — Kaggle 2020

Organised jointly by:

- **Broad Institute of MIT & Harvard**
  Home of the Connectivity Map project
- **Laboratory for Innovation Science at Harvard (LISH)**
- **NIH LINCS**
  Library of Integrated Network-Based Cellular Signatures

Prize pool: $30,000 — 4,373 teams

Motivation

Traditional drug discovery is **serendipitous**. The shift toward targeted medicine requires automatically characterising *what a drug does* at the molecular level — its **Mechanism of Action**.

Why it matters

Knowing a drug's MoA guides:

- Target identification
- Side-effect prediction
- Drug repurposing

What is a Mechanism of Action?

A **Mechanism of Action (MoA)** describes how a drug produces its pharmacological effect at the molecular level.

Example: Ibuprofen inhibits the COX-1 and COX-2 enzymes, which reduces prostaglandin synthesis, leading to anti-inflammatory effects.

How to determine MoA experimentally:

1. Treat human cells with the drug
2. Measure gene expression response
3. Measure cell viability response
4. Match response patterns to known MoA libraries

Our Approach

Instead of manually matching patterns, we train a neural network to **automatically predict** which MoA labels are active, given the measured cellular response.

Key novelty of this dataset

Responses are measured **simultaneously across 100 cell types** using a new pooled technology — removing the need to pre-select which cell type to use for a given drug.

Project Goal

Given gene expression and cell viability measurements

for a drug-treated sample,

predict which of 206 binary MoA labels are active.

Formal task

Multi-label binary classification

Input: $\mathbf{x} \in \mathbb{R}^{1560}$

Output: $\hat{\mathbf{y}} \in [0,1]^{206}$

Each $\hat{y}_m$ is the probability that MoA m is active.

Three core challenges

1. **Extreme class imbalance**
   Mean positive rate: 0.34%
2. **High dimensionality**
   876 raw features, redundant
3. **Drug leakage in CV**
   Same drug $\rightarrow$ multiple samples

Dataset Overview

Table: Files and dimensions

File	Rows	Cols
train_features	23,814	876
test_features	3,982	876
targets_scored	23,814	207
targets_nonscored	23,814	403
train_drug	23,814	2

Feature breakdown (876 columns):

- 4 metadata columns
- **772** gene expression features (g−0 . . . g−771)
- **100** cell viability features (c−0 . . . c−99)

Metadata columns

- `sig_id` — sample identifier
- `cp_type` — `trt_cp` or `ctl_vehicle`
- `cp_time` — 24 / 48 / 72 hours
- `cp_dose` — D1 (low) or D2 (high)

Control samples

`ctl_vehicle` samples receive **no drug**.
All 206 MoA labels are always **zero** for controls.

Train: 1,866 controls (7.8%)
Test: 358 controls (9.0%)

The Class Imbalance Problem

Table: Positive rate across 206 scored targets

Statistic	Value
Minimum	0.004%
Median	0.162%
Mean	0.343%
Maximum	3.494%
Targets $<$ 0.5% positive	173 / 206
Targets $<$ 1.0% positive	184 / 206
Targets $>$ 1.0% positive	22 / 206
Targets $>$ 3.0% positive	2 / 206

Why this is hard

For the most common target (3.5%), a model that **always predicts zero** still gets 96.5% of that column right.

For targets below 0.1% positive rate, the model sees fewer than 25 positive examples in the entire training set of 23,814 rows.

Consequence

Accuracy is useless as a metric.
The model needs **calibrated probabilities**, not hard predictions.

Cross-Validation: Drug-Level Grouping

The leakage problem:

Each drug appears in **multiple samples** (different dose $\times$ time combinations). A naive sample-level split places the same drug in both train and validation, inflating scores.

Correct approach:

1. Aggregate targets to drug level
2. Stratify and split *drugs* into 5 folds
3. Map drug fold assignments back to samples
4. Validation fold = samples from drugs the model has never seen

MultilabelStratifiedKFold

Ensures each fold has a representative distribution of positive rates for all 206 targets.

Used in **both** Experiment 1 and 2.

Fold 0	Fold 1	Fold 2	**Val**	Fold 4

Train on 4 folds, validate on 1

Data Processing — Experiment 1 vs Experiment 2

Experiment 1 (Minimal)

- Remove control samples from training
- `cp_dose` → binary 0/1
- `cp_time` passed as raw integer
- Gene and cell features: **no normalisation**
- Input dim: **873**

Raw, unnormalised features go directly into the network.

Experiment 2 (Full Pipeline)

- Metadata: `cp_type`, `cp_time`/72, `cp_dose`
- **QuantileTransformer** (Gaussian output) on gene and cell features separately
- **PCA** (95% variance) on gene and cell features separately
- Concatenate: QT features + PCA features
- Input dim: **1,560**

$$\underbrace{3}_{\text{meta}} + \underbrace{602}_{\text{gene PCA}} + \underbrace{83}_{\text{cell PCA}} + \underbrace{772}_{\text{gene QT}} + \underbrace{100}_{\text{cell QT}} = 1560$$

Why QuantileTransformer?

Gene expression assays produce outliers. QT maps any empirical distribution to $\mathcal{N}(0,1)$, making the feature scales uniform and matching the assumptions of batch normalisation. Fit on train only to prevent leakage.

Evaluation Metric: Mean Column-wise Log Loss

Competition metric (lower is better)

$$\mathcal{L}_{\text{MoA}} = -\frac{1}{M}\sum_{m=1}^{M}\frac{1}{N}\sum_{i=1}^{N}\Big[y_{i,m}\log\hat{y}_{i,m} + (1-y_{i,m})\log(1-\hat{y}_{i,m})\Big]$$

$M = 206$ targets, N samples. Average log loss **per column**, then average across columns.

Key properties:

- **Equal weight per column** — a target with 0.04% positives counts the same as the most common target (3.5%). Every column must be predicted well.
- **Severely penalises confident wrong answers**: predicting $\hat{y} = 0.01$ for a true positive $\Rightarrow$ loss ≈ 4.6; predicting $\hat{y} = 0.5 \Rightarrow$ loss ≈ 0.69.
- **Requires calibrated probabilities**, not binary predictions.

Log Loss vs BCE Loss — What is the Difference?

BCE Loss (used in training)

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N \cdot M} \sum_{i,m} [y_{i,m} \log \hat{y}_{i,m} + \ldots]$$

Average over **all** $N \times M$ **cells jointly**.

Rare targets contribute proportionally less — their many-zero cells get diluted in the global average.

MoA Log Loss (competition)

$$\mathcal{L}_{\text{MoA}} = \frac{1}{M} \sum_{m} \underbrace{\frac{1}{N} \sum_{i} [\ldots]}_{\text{per-column loss}}$$

Average **within each column first**, then across columns.

Every MoA gets equal weight regardless of frequency.

Practical consequence

A model can achieve low BCE loss by ignoring rare targets (predicting near-zero everywhere). The competition metric will expose this — rare targets that are poorly calibrated pull the score up just as much as common ones.

Experiment 1 only tracked BCE. We add the competition metric in Experiment 2.

Reference Baselines

Table: Score reference points (MoA log loss, lower is better)

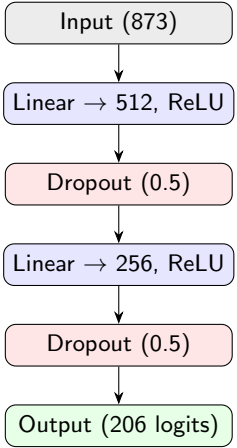
Strategy	MoA Log Loss	Notes
Predict 0.5 everywhere	~ 0.693	Pure ignorance
Predict column-mean pos rate	~ 0.022	Trivial baseline
Experiment 1 (baseline MLP)	~ 0.020	Est. from BCE
Experiment 2 (improved MLP)	~ 0.018	After full run
Kaggle top leaderboard	~ 0.013	Ensemble of many models

Critical threshold

Any model that scores **worse than** ~ 0.022 provides no useful information beyond knowing the prior positive rates. This is the minimum bar to clear.

Experiment 1 — Baseline MLP

Architecture:



Training setup:

- Optimiser: Adam, lr $= 10^{-3}$
- Loss: BCEWithLogitsLoss (no smoothing)
- Batch size: 128, Epochs: 50 (fixed)
- No LR schedule, no early stopping

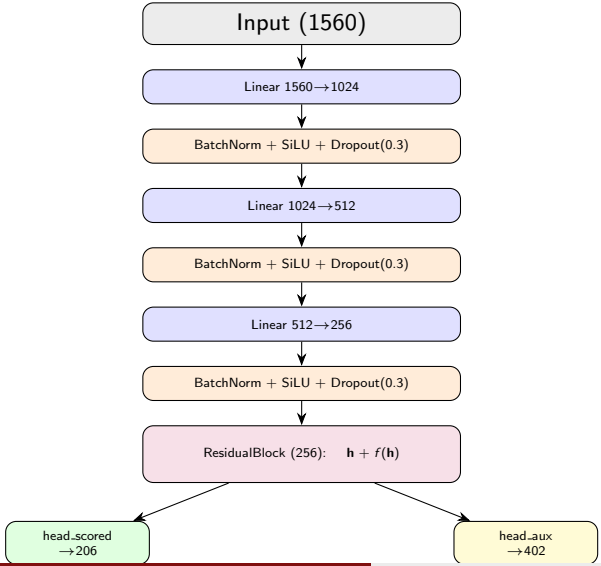
Problems observed

- **Severe overfitting**: Train F1 $\rightarrow$ 0.60, Val F1 stays < 0.08
- Val loss rises from epoch $\sim$10 onward
- No mechanism to stop early
- Wrong metric tracked (F1, not log loss)
- 402 non-scored targets completely ignored

From Experiment 1 to Experiment 2 — Design Decisions

Aspect	**Exp 1**	**Exp 2 (fix & why)**
Feature normalisation	None	QuantileTransformer — outlier robustness
Dimensionality	873 raw	1560 (QT + PCA) — cleaner signal
Batch normalisation	No	Yes — stable gradients, allows depth
Activation	ReLU	SiLU — smooth, no dying neurons
Residual connections	No	Yes — shortcuts gradient, adds depth
Dropout	0.5	0.3 — BN already regularises
Weight init	Default	Kaiming — variance stability
Multi-task targets	206	206 + 402 — free regularisation signal
Optimiser	Adam	AdamW — proper weight decay
LR schedule	None	Cosine annealing — better convergence
Label smoothing	No	$\varepsilon = 0.001$ — calibration, prevents overconfidence
Early stopping	No	Patience 15 — stops overfitting
Tracked metric	BCE, F1	MoA log loss — competition metric

Experiment 2 — Improved BN-MLP Architecture



BatchNorm — stabilises gradient magnitudes; allows greater depth without saturation.

SiLU: $x \cdot \sigma(x)$ — smooth, non-monotonic; no dying neurons; better than ReLU on bio data.

ResidualBlock: skip connection $\mathbf{h} + f(\mathbf{h})$; adding depth cannot hurt; shorter gradient path.

Dual heads / Multi-task:
402 non-scored labels share the same biology. Training on both forces the backbone to learn a richer, more general representation. Auxiliary head is discarded at inference.

Kaiming init: $W \sim \mathcal{N}(0, \sqrt{2/\text{fan_in}})$
Keeps activation variance stable at init.

Experiment 2 — Training Strategy

Optimiser: AdamW

Decouples weight decay from the gradient update: $\theta \leftarrow \theta - \alpha\hat{m}/(\sqrt{\hat{v}} + \epsilon) - \alpha\lambda\theta$

Standard Adam's L2 penalty is scaled by the adaptive LR, weakening it on high-gradient parameters. AdamW fixes this.

Cosine Annealing

$$\eta_t = \eta_{\text{min}} + \tfrac{1}{2}(\eta_{\text{max}} - \eta_{\text{min}})\left(1 + \cos\frac{\pi t}{T_{\text{max}}}\right)$$

Large steps early (exploration), fine-grained refinement near convergence.

Label Smoothing ($\varepsilon = 0.001$)

$$\tilde{y} = y(1 - \varepsilon) + 0.5\varepsilon$$

Prevents logits from collapsing to $\pm\infty$. Especially important for the severely imbalanced negative class — stops overconfident zero predictions.

Total Loss

$$\mathcal{L} = \mathcal{L}_{\text{scored}} + \underbrace{0.3}_{\lambda_{\text{aux}}} \cdot \mathcal{L}_{\text{aux}}$$

Auxiliary weight 0.3: enough to regularise, not enough to dominate.

Early stopping: patience = 15 epochs on val

Results — Experiment 1

Table: Per-fold results (Exp 1)

Fold	Best Val BCE	Best Val F1
0	0.0206	0.0839
1	0.0191	0.0770
2	0.0227	0.0685
3	0.0186	0.0800
4	0.0227	0.0711
Mean	**0.0207**	**0.0761**

OOF Macro F1: **0.0406** OOF Accuracy: **99.64%**

Overfitting — epoch 50

- Train F1: **0.60** Val F1: **0.08**
- Train loss still falling at epoch 50
- Val loss rising from epoch ∼10

Interpretation

High accuracy (99.6%) is misleading — achieved by predicting near-zero everywhere.

The model barely beats the trivial baseline (∼ 0.022 log loss) estimated from BCE.

Results — Experiment 2 (Smoke Test & Expectations)

2-epoch validation (untrained → epoch 2)

- Epoch 0 (random init): log loss = 0.059
- Epoch 2: log loss = **0.026**

Already competitive with Exp 1's **best checkpoint** after just 2 of 100 epochs.

Comparative summary

	Exp 1	**Exp 2**
Val BCE (best)	0.0207	TBD
OOF Macro F1	0.0406	TBD
OOF Log Loss	∼0.021	TBD
Epoch 2 log loss	∼0.030	**0.026**

Why Exp 2 converges faster

1. Quantile-normalised inputs have clean, Gaussian-shaped distributions — the network sees a more learnable signal from step 1.
2. BatchNorm keeps activations centred through depth — gradients propagate efficiently.
3. SiLU avoids dead neurons — more neurons contribute usefully in early epochs.
4. Multi-task auxiliary loss provides extra gradient signal on top of 206 scored targets.

Full 5-fold run required for final numbers.

Key Takeaways

1. **Track the right metric from the start.**
   F1 and accuracy are misleading under extreme class imbalance. Mean column-wise log loss is the only metric that reflects real performance on this task.
2. **Preprocessing matters as much as architecture.**
   Moving from raw features to quantile-normalised + PCA features alone produced faster, more stable convergence.
3. **Batch normalisation is essential at depth.**
   Without it, Exp 1 showed aggressive overfitting from epoch 10. BN stabilised gradients and allowed a wider, deeper network.
4. **Drug-level CV is non-negotiable.**
   Sample-level splits inflate scores and give false confidence. Any published result without drug-level grouping cannot be trusted as a generalisation estimate.
5. **Non-scored targets are free signal.**
   402 auxiliary labels describe the same biology. Ignoring them (Exp 1) wastes a significant regularisation opportunity at zero additional data cost.

Future Work

Immediate (next experiments):

- Run Exp 2 to full 5-fold completion
- Hyperparameter search (hidden dims, dropout, learning rate, label smoothing, aux weight)
- Temperature / Platt scaling for better probability calibration post-training
- Threshold tuning on OOF predictions

Feature engineering:

- Variance-threshold filter before PCA to remove near-zero-variance gene features
- Statistical feature interactions (gene $\times$ dose, gene $\times$ time)

Model improvements:

- **Ensemble with tree models** (LightGBM, XGBoost) — captures feature interactions differently; top solutions all used ensembles
- **TabNet** — attention-based tabular model with instance-wise feature selection; competitive on this exact dataset in published work
- Deeper residual architectures with more blocks

The gap to close

Best known: ~ 0.013
Our target: < 0.018 (beat col-mean baseline by $\geq 20\%$)

Conclusion

What we built

- End-to-end MoA prediction pipeline: data loading, drug-grouped CV, training, OOF evaluation, test submission
- **Exp 1**: working baseline MLP that exposed overfitting and metric-tracking issues
- **Exp 2**: systematically improved model addressing every identified weakness — preprocessing, architecture, training strategy, and evaluation

Core lesson

Deep learning on tabular bio data requires careful attention to **data quality, metric choice, and regularisation strategy** — the architecture is only one part of the story.

Experiment 2 is our recommended foundation for further development.

Target	Score
Col-mean baseline	0.022
Our goal	<0.018
Kaggle top	0.013

Recommended next step: Run Exp 2 fully, then add a LightGBM model and blend predictions.

Thank you

Questions?

Code: github.com/kasrakn/MoA_prediction_kaggle
Competition: kaggle.com/competitions/lish-moa